

Correção do Simulado

COMP0496 — Programação A — Turma T05

Departamento de Computação

Universidade Federal de Sergipe

Maio de 2026

Q1 — Triagem de Risco (2 pt)

Q2 — Plantão de Cirurgias (2 pt)

Q3 — Histórico do Paciente (2 pt)

Q4 — Atendimento na UTI (4 pt)

Erro Recorrente: Dados Hardcoded

Q1 — Triagem de Risco (2 pt)

Triagem de Risco (2 pt)

A triagem hospitalar classifica cada paciente com base em **pressão sistólica** (real, escala mmHg/10) e **idade** (inteiro, em anos). Classificações:

- **NORMAL**: pressão e idade dentro dos limites saudáveis.
- **BAIXO**: pressão acima de 120 mmHg, sem outros agravantes.
- **MEDIO**: pressão acima de 140 mmHg **ou** paciente com mais de 60 anos.
- **ALTO**: pressão acima de 140 mmHg **e** paciente com mais de 60 anos.
- **CRITICO**: pressão inferior a 90 mmHg — emergência independente da idade.

Escreva a função `risco(pressao, idade)` que retorna a classificação.

O programa lê *N*; para cada paciente lê nome, pressão e idade. Imprime `<nome>`:
`<classificacao>`.

Erro 1: Ordem incorreta das verificações

Alguns alunos verificaram MEDIO **antes** de ALTO. Como MEDIO usa OR (pressão > 140 **ou** idade > 60), ele **absorve** todos os casos de ALTO, que nunca é alcançado (dead code).

```
1  # ERRADO: ALTO nunca sera alcancado
2  if pressao > 14 or idade > 60:      # MEDIO
3      return "MEDIO"
4  elif pressao > 14 and idade > 60:    # ALTO (dead code!)
5      return "ALTO"
```

Erro 2: CRITICO verificado por último

Outro erro foi checar CRITICO (pressão < 90 mmHg) **depois** de BAIXO (pressão > 120 mmHg). Uma pressão de 85 mmHg não passa pela condição BAIXO e cai no else/NORMAL, nunca chegando ao CRITICO.

Erro 3: Escala da pressão mal interpretada

O enunciado informa que a pressão vem na escala **mmHg/10** (ex: 14.0 = 140 mmHg). Um aluno multiplicou por 10 novamente ($P = \text{pressao} * 10$), obtendo valores como 1400 — completamente fora dos limites.

```
1 # ERRADO: duplica a escala
2 P = pressao * 10    # entrada 14.0 -> P = 140.0 (correto seria usar 14.0 direto)
3 if P > 140: ...     # isso so funciona "por acidente" neste caso
```

Erro 4: Assinatura da função alterada

O enunciado pede `risco(pressao, idade)`, mas alguns alunos adicionaram `nome` como parâmetro extra: `risco(nome, pressao, idade)`. Isso viola a especificação.

Erro 5: print dentro da função + print na chamada

A função faz `print` internamente e retorna `None`. Ao chamar `print(risco(...))`, imprime o resultado e depois `None`.

Solução correta

```
1 def risco(pressao, idade):
2     pressao_mmhg = pressao * 10
3     if pressao_mmhg < 90:
4         return "CRITICO"
5     elif pressao_mmhg > 140 and idade > 60:
6         return "ALTO"
7     elif pressao_mmhg > 140 or idade > 60:
8         return "MEDIO"
9     elif pressao_mmhg > 120:
10        return "BAIXO"
11    else:
12        return "NORMAL"
13
14 n = int(input())
15 for _ in range(n):
16     nome = input()
17     pressao = float(input())
18     idade = int(input())
19     print(f"{nome}: {risco(pressao, idade)}")
```

Ponto-chave

A ordem correta é: CRITICO → ALTO → MEDIO → BAIXO → NORMAL.

Regra geral: verificar **do mais restritivo para o mais genérico**.

Q2 — Plantão de Cirurgias (2 pt)

Plantão de Cirurgias (2 pt)

O hospital registra a duração (em minutos) de cada cirurgia do plantão. A digitação termina com **0**, que NÃO conta. Usar `while` com sentinela.

Cada cirurgia é classificada por duração:

- $\leq 30 \rightarrow$ **RAPIDA**
- $\leq 90 \rightarrow$ **MEDIA**
- $> 90 \rightarrow$ **LONGA**

Imprima 5 linhas:

- RAPIDAS: `<r>`, MEDIAS: `<m>`, LONGAS: `<l>`
- TOTAL: `<total>`, MEDIA: `<m>` (1 casa decimal)
- Se não houver cirurgia, imprimir zeros e MEDIA: `0.0`.

Erro 1: Variável não inicializada

Um aluno usou a variável `quantidade` para contar cirurgias, mas **nunca a inicializou** com `quantidade = 0`. Resultado: `NameError` imediato ao executar.

```
1 # ERRADO: variavel nunca declarada
2 while tempo != 0:
3     quantidade += 1      # NameError: name 'quantidade' is not defined
4     ...
```

Erro 2: Média recalculada a cada iteração

Alguns alunos calcularam `MEDIA = total / qtd` **dentro** do loop, a cada cirurgia. Funciona, mas é desnecessário e ineficiente — basta calcular uma vez, após o loop.

Erro 3: Divisão por zero sem proteção

Se o primeiro valor digitado for 0 (nenhuma cirurgia), calcular `total / qtd` gera `ZeroDivisionError`. É preciso verificar `if qtd == 0` antes.

Solução correta

```
1  rapidas = 0
2  medias = 0
3  longas = 0
4  total = 0
5  qtd = 0
6
7  duracao = int(input())
8  while duracao != 0:
9      qtd += 1
10     total += duracao
11     if duracao <= 30:
12         rapidas += 1
13     elif duracao <= 90:
14         medias += 1
15     else:
16         longas += 1
17     duracao = int(input())
18
19 print(f"RAPIDAS: {rapidas}")
20 print(f"MEDIAS: {medias}")
21 print(f"LONGAS: {longas}")
22 print(f"TOTAL: {total}")
23 if qtd == 0:
24     print("MEDIA: 0.0")
25 else:
26     print(f"MEDIA: {total / qtd:.1f}")
```

Q3 — Histórico do Paciente (2 pt)

Histórico do Paciente (2 pt)

Um paciente teve a temperatura corporal aferida N vezes (reais). Leia N e os valores, **armazene numa lista** e imprima:

- MEDIA: `<m>` (2 casas decimais)
- FEBRE: `<q>` (medições > 37.5)
- HIPOTERMIA: `<q>` (medições < 36.0)
- PRIMEIRAS: `<3 primeiras>`
- INVERSA: `<lista invertida>`

Entrada:

```
5
36.5  38.0  37.8  35.5
36.2
```

Saída:

```
MEDIA: 36.80
FEBRE: 2  HIPOTERMIA: 1
PRIMEIRAS: [36.5, 38.0, 37.8]
INVERSA: [36.2, 35.5, 37.8, 38.0, 36.5]
```

Erro 1: `[:-1]` em vez de `::-1]`

Um aluno usou `temps[:-1]` para inverter a lista. Porém:

- `temps[:-1]` → remove o **último** elemento (fatiamento até o penúltimo)
- `temps::-1]` → **inverte** a lista (passo `-1`)

Um único caractere de diferença que muda completamente o resultado!

Erro 2: `[:4]` em vez de `[:3]` para PRIMEIRAS

O enunciado pede as **3 primeiras** medições. Um aluno usou `l[:4]`, retornando **4** elementos.

Erro 3: Rótulo errado na saída

Um aluno imprimiu `INVERSAS:` em vez de `INVERSA:`. A saída deve ser **exatamente** como especificada no enunciado.

Solução correta

```
1 n = int(input())
2 temps = []
3 for _ in range(n):
4     temps.append(float(input()))
5
6 media = sum(temps) / n
7 febre = 0
8 hipo = 0
9 for t in temps:
10     if t > 37.5:
11         febre += 1
12     if t < 36.0:
13         hipo += 1
14
15 print(f"MEDIA: {media:.2f}")
16 print(f"FEBRE: {febre}")
17 print(f"HIPOTERMIA: {hipo}")
18 print(f"PRIMEIRAS: {temps[:3]}")
19 print(f"INVERSA: {temps[::-1]}")
```

Atenção aos detalhes

- `[:3]` = primeiros 3 `[::-1]` = lista invertida
- FEBRE e HIPOTERMIA usam `if` independentes (não `elif`), pois são mutuamente exclusivos por natureza, mas conceitualmente são contagens separadas.

Q4 — Atendimento na UTI (4 pt)

Atendimento na UTI (4 pt)

N pacientes têm cada um V variáveis vitais avaliadas (escala 1–10; quanto maior, mais grave) numa matriz $N \times V$. Leia N , V , depois N nomes e a matriz.

- Calcule a **gravidade total** de cada paciente (soma da linha).
- Ordene em ordem **decrescente** de gravidade usando **Insertion Sort manual**.
- Imprima `<pos>. <nome> (gravidade <total>)`.

Restrição

É **proibido** usar `sort()`, `sorted()`, `max()` ou `min()` para ordenar.

Exemplo de saída:

1. Beatriz (gravidade 32)
2. Carla (gravidade 20)
3. Adriana (gravidade 14)

Erro 1: Variável `V` lida e nunca utilizada

Vários alunos leram `N, V = map(int, input().split())` mas usaram `sum(input().split())` sem limitar a `V` colunas. Funciona para entradas bem-formadas, mas indica leitura mecânica do enunciado.

Erro 2: Nomes lidos com `input().split()` em linha única

O enunciado fornece **um nome por linha**. Ler todos os nomes com `nomes = input().split()` em uma única linha só funciona para nomes simples sem espaço — e diverge do formato de entrada.

Erro 3: Questão não entregue

Um(a) aluno(a) não entregou a Q4, perdendo os 4 pontos da questão mais valiosa da prova.

O **Insertion Sort** percorre o array da esquerda para a direita. Para cada elemento, insere-o na posição correta em relação aos já ordenados.

Para ordenação **decrescente**, desloca enquanto o elemento à esquerda for **menor** que a chave:

```
1  # Insertion Sort decrescente
2  for i in range(1, n):
3      chave = gravidades[i]
4      j = i - 1
5      while j >= 0 and gravidades[j] < chave: # < para decrescente
6          gravidades[j + 1] = gravidades[j]
7          j -= 1
8      gravidades[j + 1] = chave
```

Cuidado com arrays paralelos

Ao ordenar `gravidades[]`, é preciso mover `nomes[]` em sincronia. Cada troca/deslocamento em `gravidades` deve ser replicada em `nomes`.

Solução correta

```
1 n, v = map(int, input().split())
2 nomes = []
3 for _ in range(n):
4     nomes.append(input())
5
6 gravidades = []
7 for i in range(n):
8     valores = list(map(int, input().split()))
9     gravidades.append(sum(valores))
10
11 # Insertion Sort decrescente
12 for i in range(1, n):
13     chave_g = gravidades[i]
14     chave_n = nomes[i]
15     j = i - 1
16     while j >= 0 and gravidades[j] < chave_g:
17         gravidades[j + 1] = gravidades[j]
18         nomes[j + 1] = nomes[j]
19     j -= 1
20     gravidades[j + 1] = chave_g
21     nomes[j + 1] = chave_n
22
23 for i in range(n):
24     print(f"{i + 1}. {nomes[i]} (gravidade {gravidades[i]})")
```

Erro Recorrente: Dados Hardcoded

Problema grave detectado em várias questões

Alguns alunos escreveram os dados de teste **diretamente no código**, sem usar `input()` para ler a entrada. O programa só funciona para aqueles valores específicos e não aceita nenhuma outra entrada.

```
1  # ERRADO: dados fixos no código
2  nome = "Antonio"
3  pressao = 15.0
4  idade = 70
5
6  # ERRADO: lista fixa em vez de input()
7  entradas = [25, 45, 120, 60, 180, 20, 0]
```

```
1  # CORRETO: leitura dinâmica com input()
2  N = int(input())
3  for _ in range(N):
4      nome = input()
5      pressao = float(input())
6      idade = int(input())
```

Ler apenas um caso em vez de N

Alguns alunos leram apenas **um paciente/caso** com `input()`, mas sem o loop para iterar sobre N entradas. O enunciado pede para ler N e processar N pacientes.

Exemplo: Q1 pede N pacientes

- **Errado:** ler um único nome, pressão e idade, sem loop
- **Correto:** `N = int(input())` seguido de `for _ in range(N):`

Ler nomes em linha única em vez de um por linha

Em Q4, alguns alunos leram todos os nomes com `nomes = input().split()` em uma única linha, mas o enunciado fornece **um nome por linha**. Isso falharia com nomes compostos.

Por que isso importa?

O programa deve funcionar para **qualquer entrada válida**, não apenas para os dados de exemplo do enunciado. Um programa com dados fixos não é um programa — é uma simulação de saída.

Erros mais frequentes

1. **Dados hardcoded:** usar `input()` para ler dados — nunca fixar valores no código
2. **Leitura incompleta:** ler N e iterar com `for`, não ler apenas 1 caso
3. **Ordem dos `if/elif`:** verificar do mais restritivo ao mais genérico
4. **Variáveis não inicializadas:** sempre declarar contadores antes do loop
5. **Fatiamento:** $[:-1] \neq [::-1]$ — atenção ao passo negativo
6. **Assinatura de funções:** respeitar exatamente o que o enunciado pede
7. **Formato de saída:** maiúsculas, pontuação e casas decimais importam

Boas práticas observadas

- Nomes de variáveis descritivos (`chave_gravidade`, `hipotermia`)
- Proteção contra divisão por zero
- Uso correto de f-strings com formatação numérica